

SAGE: Self-evolving Agents with Reflective and Memory-augmented Abilities

Xuechen Liang^{1*}, Yangfan He^{2*}, Yinghui Xia^{3*}, Xinyuan Song⁴,
Jianhui Wang⁵, Meiling Tao⁶, Li Sun⁷, Xinhang Yuan⁸, Jiayi Su⁹,
Keqin Li¹⁰, Jiaqi Chen¹⁰, Jinsong Yang^{3*}, Siyuan Chen¹⁰, Tianyu Shi^{11†}

¹ East China Jiao Tong University, ² University of Minnesota - Twin Cities,
³ AutoAgents.ai, ⁴ Emory University,

⁵ University of Electronic Science and Technology of China,

⁶ Guangdong University of Technology,

⁷ Amazon, ⁸ Washington University, ⁹ Xiamen University,

¹⁰ Independent Researcher, ¹¹ University of Bristol, ¹² University of Toronto
Project Page: <https://jianhuiwemi.github.io/SAGE>

Abstract

Large language models (LLMs) have made significant advances in the field of natural language processing, but they still face challenges such as continuous decision-making, lack of long-term memory, and limited context windows in dynamic environments. To address these issues, this paper proposes an innovative framework—Self-evolving Agents with Reflective and Memory-augmented Abilities (**SAGE**). The SAGE framework comprises three agents: the User, the Assistant, and the Checker. By integrating iterative feedback, reflective mechanisms, and a memory optimization mechanism based on the Ebbinghaus forgetting curve, it significantly enhances the agents’ capabilities in handling multi-tasking and long-span information. The agents, through self-evolution, can adaptively adjust strategies, optimize information storage and transmission, and effectively reduce cognitive load. We evaluate the performance of the SAGE framework on multiple benchmarks and long text tasks. Experimental results show that SAGE significantly improves model performance, achieving a 2.26X improvement on closed-source models and an improvement ranging from 57.7% to 100% on open-source models, with particularly notable effects on smaller models.

1 Introduction

In recent years, large language models (LLMs) have made significant progress in the field of natural language processing, demonstrating powerful performance in tasks such as dialogue and text generation [Brown et al. \(2020\)](#); [He et al. \(2025; 2024\)](#). Recently, there has been growing interest in applying LLMs as autonomous agents (LLM agents), which use language not only for understanding and generation, but also for planning and acting in interactive environments [Yao et al. \(2023b\)](#); [Shinn et al. \(2023\)](#); [Liang et al. \(2024\)](#); [Li et al. \(2024\)](#); [Zhou et al. \(2024\)](#). However, these models still face several challenges: (1) LLM Agents need to continuously make decisions in changing environments and adapt to new situations and tasks. (2) LLM Agents lack long-term memory mechanisms, which is increasingly evident in situations requiring sustained interaction with the environment [Graves et al. \(2016\)](#). The limited context window also hinders the model’s ability to handle information over long time spans [Rae et al. \(2019\)](#).

To tackle these challenges, researchers have proposed meta-learning and multi-task learning to enhance the transferability and adaptability of LLM agents. For memory limitations, prior works like MemGPT [Packer et al. \(2024\)](#) use a FIFO queue to manage forgetting,

*Equal contribution.

†Corresponding author: ty.shi@mail.utoronto.ca

while MemoryBank employs a forgetting curve based on insertion time. However, these approaches are often task-specific, lacking a general framework to systematically improve LLM agents in complex environments. Recent innovations, such as AutoGPT [Yang et al. \(2023\)](#) and BabyAGI [Nakajima \(2024\)](#), leverage LLMs as core controllers, aiming to solve real-world challenges. Yet, multi-agent frameworks still face issues like communication overload, heavily relying on memory to maintain context. As interaction history grows, resource demands and latency increase, limiting efficient deployment in practical scenarios.

In this paper, we propose an innovative framework, Self-evolving Agents with reflective and memory-augmented abilities (SAGE). By enhancing agents’ self-adjustment capabilities through reflection, they can more effectively utilize historical information and make efficient decisions when faced with complex and dynamic tasks. From the perspective of self-evolution, we introduce a memory optimization mechanism based on the Ebbinghaus forgetting curve [Ebbinghaus \(1885\)](#). This mechanism helps agents selectively retain key information, optimize information storage and transmission, reduce unnecessary cognitive load, and enhance agents’ capabilities in interaction tasks with the environment. Experimental results demonstrate that our approach consistently enhances the performance of both proprietary and open-source LLMs across a wide range of benchmarks. The improvements are especially notable in smaller models, where the gains are more pronounced. On tasks such as multi-source question answering and code generation, our method sets a new standard, outperforming existing techniques and achieving leading benchmarks [Etezadi & Shamsfard \(2023\)](#), including AgentBench [Liu et al. \(2023b\)](#).

The main contributions of our work are as follows:

- We propose a novel framework, SAGE, which introduces a reflection mechanism to enhance agents’ self-adjustment capabilities. Without any additional training, this enables agents to utilize historical information more effectively and make better decisions when faced with complex and dynamic tasks.
- We introduce a memory optimization mechanism based on the Ebbinghaus forgetting curve. This helps agents selectively retain key information, reducing the issue of information overload in multi-agent systems.
- SAGE achieves improvements over strong baselines in multiple challenging real-world tasks and attains state-of-the-art results on benchmarks. This framework can be applied to other LLMs, with particularly strong improvements in smaller models.

2 Related work

2.1 Self-Improvement of Reasoning and Decision-Making

Deep learning has transformed multiple domains including NLP, time series analysis and computer vision [Qiu et al. \(2025a;b; 2024\)](#). A lot of research is focused on making large language models (LLMs) better at improving themselves. Some researchers are working on using carefully crafted prompts to help models learn how to get better, although this usually only works for one-off tasks. Others are tweaking how models get feedback during tasks, which helps them get better at thinking things through [Huang et al. \(2022\)](#). There’s also work on using strategies like random beam searches to help models make smarter decisions and assess their own work. Most current methods rely on quick, one-off tweaks and learning strategies that need lots of resources and hands-on tech help [Tian et al. \(2024\)](#). This paper introduces a self-reflection mechanism, showing that LLMs can keep getting better and produce higher quality work across different tasks, all without needing extra training.

2.2 Memory Mechanism for LLM-based Agents

In LLM-based agents, the memory module stores, processes, and retrieves task-related information, supporting knowledge accumulation, experience handling, and decision-making. To enhance the self-evolution capabilities of these agents, researchers are focused on designing and optimizing these memory modules [Raffel et al. \(2020\)](#). Past research has covered various designs and implementations of memory modules. This includes integrating information from different trials to boost reasoning abilities or storing information in natural

language to enhance the module’s interpretability and user-friendliness Wada et al. (2019). Despite progress, self-adjustment and memory management still need improvement to handle complex real-world problems more effectively.

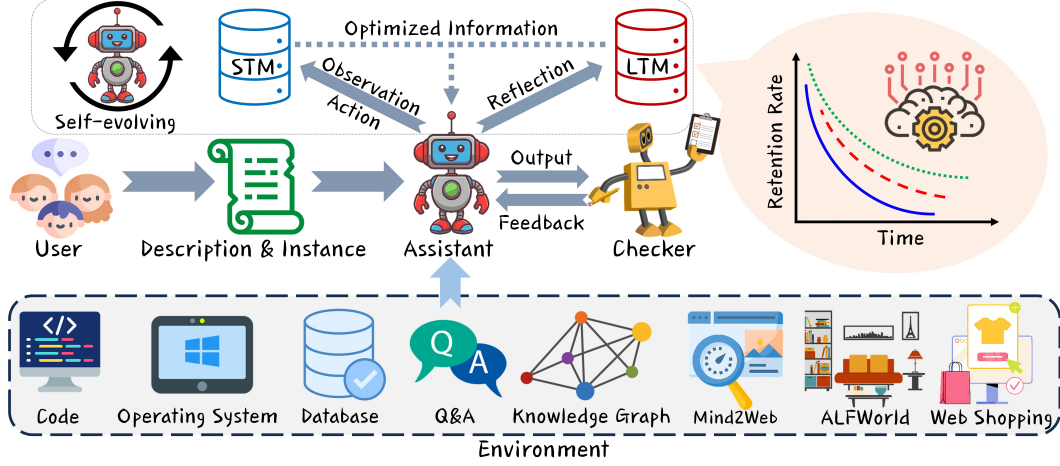


Figure 1: An illustration of the SAGE: a user provides a description and instance to the assistant with short-term (STM) and long-term (LTM) memory. The assistant performs observation, action, reflection, and output, which the checker reviews. The retention rate curve on the right illustrates memory decay over time, with a self-evolving loop guiding continued updates.

3 Method

In this section, we present the SAGE framework, designed to improve agent performance by leveraging three core mechanisms: iterative feedback, reflection, and MemorySyntax (as shown in Figure 1). The assistant agent A iteratively updates its policy π_θ based on feedback f_t provided by the checker agent C , optimizing over successive iterations to maximize the expected reward R . The reflection mechanism allows A to incorporate historical observations $\mathcal{O}_t$ and actions $\mathbf{a}_t$, forming a self-reflection r_t , which is stored in the memory $\mathcal{M}_L$ for future decision-making. Finally, MemorySyntax combines the Ebbinghaus forgetting curve with linguistic principles to manage memory decay, dynamically updating the agent’s short-term memory $\mathcal{M}_S$ and long-term memory $\mathcal{M}_L$ by prioritizing information based on its retention strength $S(I_t)$, thus improving the agent’s ability to retain crucial information while discarding less relevant data. The subsequent subsections detail these components.

3.1 Iterative Feedback

The iterative feedback mechanism in the SAGE framework enables the assistant agent A to refine its policy π_θ through repeated interactions with the checker agent C . At each iteration t , the assistant receives feedback f_t based on its current output $\mathbf{o}_t$, and adjusts its policy accordingly. This process continues until the checker validates the output or the iteration cap N is reached, ensuring that A incrementally optimizes its decisions to improve task performance over successive iterations.

3.1.1 Initialization Phase

Role Assignment. In the SAGE framework, three agents are introduced: the user U , the assistant A , and the checker C . The user, upon receiving prompt P_U , assumes the role of task proposer by specifying a task $\mathcal{T}_U$ and related constraints $\mathcal{C}_U$. The assistant, upon receiving prompt P_A , generates a sequence of actions $\mathbf{a}_t$ based on the observations $\mathcal{O}_t$ and environment $\mathcal{E}$. The checker C evaluates the output $\mathbf{o}_A$ produced by the assistant, providing feedback f_C based on the discrepancy between $\mathbf{o}_A$ and the expected result, updating its policy π_θ iteratively to minimize this gap.

Task Assignment. The task $\mathcal{T}_U$ provided by the user includes an initial task description $\mathbf{d}_U$

and an instance $\mathbf{i}_U$ that serves as the reference for correct output. This forms the input set $\mathcal{I}_A = (\mathbf{d}_U, \mathbf{i}_U)$ for the assistant to initiate its generative process. The assistant then proceeds by selecting an action $\mathbf{a}_t$ at each time step t , guided by π_θ , with the goal of maximizing the reward R_t for completing $\mathcal{T}_U$.

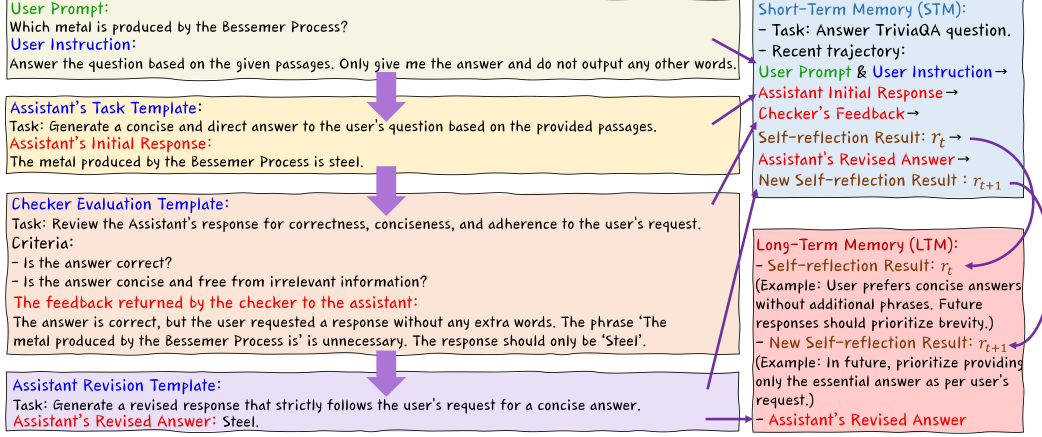


Figure 2: An example of the assistant’s iterative workflow, including checker evaluation, prompt templates for feedback, and reflection processes integrating short-term and long-term memory.

3.1.2 Actual Interaction Phase

Following the role assignment and task definition in the initialization phase, the assistant A transitions into the actual interaction phase to generate outputs aimed at accomplishing the task $\mathcal{T}_U$. In this phase, A iteratively produces outputs $\mathbf{o}_t$ at each time step t based on the task description $\mathbf{d}_U$ and instance $\mathbf{i}_U$ provided in the input set $\mathcal{I}_A = (\mathbf{d}_U, \mathbf{i}_U)$. At each time step t , the assistant selects an action $\mathbf{a}_t$ by following its policy π_θ , which is conditioned on the current state s_t , the reward signal R_t (the reward score for task performance), and feedback f_t^i from the checker C . This decision-making process is formalized as:

$$\mathbf{o}_t \sim \pi_\theta(\mathbf{o}_t \mid s_t, R_t, f_t^i), \quad (1)$$

where π_θ represents the assistant’s policy, R_t reflects the reward signal based on task performance at time t , and f_t^i is the feedback provided by the checker during the i -th iteration.

As the interaction progresses, the checker C evaluates each output $\mathbf{o}_t$ generated by A , comparing it against the expected outcome derived from $\mathbf{i}_U$. Based on this comparison, the checker provides iterative feedback f_t^i to guide A in refining its actions $\mathbf{a}_t$ and outputs $\mathbf{o}_t$. The iterative refinement continues until either the checker validates the output as correct or the iteration limit N is reached.

Theoretical optimality of iterative feedback mechanism. In the SAGE framework, the assistant repeatedly updates its policy through this checker feedback, enabling the outputs to be incrementally refined until either the result is validated or a specified iteration limit is reached. The assistant’s utility R_A reflects task performance, and the checker’s utility R_C depends on its feedback. The following theorem indicates that this iterative feedback mechanism leads to strategy stability in the sense of a Nash equilibrium [Fudenberg & Tirole \(1991\)](#).

Theorem 3.1 (Theory for the multi-agent iterative feedback system). *Let $\mathcal{U}, \mathcal{A}, \mathcal{C}$ denote the compact, convex strategy spaces of the user (U), assistant (A), and checker (C), respectively. Assume that the utility functions*

$$R_U : \mathcal{U} \times \mathcal{A} \times \mathcal{C} \rightarrow \mathbb{R}, \quad R_A : \mathcal{U} \times \mathcal{A} \times \mathcal{C} \rightarrow \mathbb{R}, \quad \text{and} \quad R_C : \mathcal{U} \times \mathcal{A} \times \mathcal{C} \rightarrow \mathbb{R} \quad (2)$$

are continuous in each player's strategy. Then, by the Debreu-Glicksberg-Fan fixed-point theorem, there exists a Nash equilibrium

$$(s_U^*, s_A^*, s_C^*) \in \mathcal{U} \times \mathcal{A} \times \mathcal{C}. \quad (3)$$

Furthermore, suppose that the assistant's policy π_θ is updated via policy gradient methods and that the checker's strategy is refined through convex optimization. Then, the iterative update procedures yield sequences

$$\{\pi_\theta^{(k)}\}_{k \geq 0} \quad \text{and} \quad \{f^{(k)}\}_{k \geq 0}, \quad (4)$$

which converge to a stable strategy profile (π_θ^*, f^*) , and has:

$$R_A(\pi_\theta^*, f^*) \geq R_A(\pi_\theta, f^*), \quad R_C(\pi_\theta^*, f^*) \geq R_C(\pi_\theta, f). \quad (5)$$

This result demonstrates that the iterative feedback mechanism enhances the model's strategy stability by converging to a Nash equilibrium in the three-player game. It provides a stronger justification for the three-agent system versus simpler alternatives (such as **two-agent systems**). For the detailed theoretical explanation and proof, see A.2.

3.1.3 Evolutionary Goals and Directions

Leveraging the feedback f_t^i obtained at each iteration t , the assistant A formulates new evolutionary objectives:

$$\mathcal{G}^{t+1} = (\mathcal{A}^{t+1}, \mathcal{D}^{t+1}), \quad \mathcal{D}^{t+1} = \arg \min_{\mathcal{D}_t \in \Delta} \sum_{i \in I_t} L_D(\mathcal{D}_t; f_t^i, \pi_\theta^t), \quad (6)$$

where $\mathcal{A}^{t+1}$ represents the updated memory optimization mechanisms, and $\mathcal{D}^{t+1} \in \Delta$ refers to the model's self-adjustments to make the RL algorithm converge. These evolutionary objectives guide the assistant in updating its policy π_θ for the subsequent iteration. The policy update is governed by the function ψ , which integrates the current policy π_θ^t with the new evolutionary objectives $\mathcal{G}^{t+1}$:

$$\theta^{t+1} = \phi(\theta^t, \mathcal{G}^{t+1}) = \theta^t + \alpha \nabla_\theta [\lambda_A L_A(\theta^t, \mathcal{A}^{t+1}) + \lambda_D L_D(\theta^t, \mathcal{D}^{t+1})]. \quad (7)$$

Here $L_A(\theta, \mathcal{A})$ and $L_D(\theta, \mathcal{D})$ are MSE loss functions corresponding to the memory-optimization and self-adjustment aspects, respectively, and $\lambda_A, \lambda_D \geq 0$ are weighting coefficients. The iterative policy refinement enables the assistant A to continuously adapt its strategies based on cumulative feedback and evolving task requirements, thereby improving its overall performance in dynamic environments.

3.2 Memory Management

The SAGE framework implements a dual-memory system, consisting of Short-Term Memory (STM) and Long-Term Memory (LTM), to manage task-relevant information and enhance the agent's reasoning and decision-making capabilities (see Figure 2 for a visual representation of this process).

Short-Term Memory (STM). STM is responsible for storing immediate, task-specific data with limited capacity. It updates rapidly with new observations ($\mathcal{O}_t$) and actions ($\mathbf{a}_t$), maintaining a recent trajectory history $\mathcal{T}_t = (\mathcal{O}_t, \mathbf{a}_t)$. This allows the agent to make real-time decisions and respond quickly to dynamic changes in the environment Mnih et al. (2015).

Long-Term Memory (LTM). LTM retains critical information and self-reflections (r_t) over extended periods, enabling the agent to accumulate knowledge from past interactions and apply it to future tasks. Stored as $\mathcal{M}_L = \{r_t \mid t \in T\}$, this memory mechanism allows the agent to use prior experiences to improve task performance, particularly in complex environments that require long-span information Graves et al. (2016).

By integrating STM and LTM, the SAGE framework allows the agent to balance immediate task demands with the ability to draw from accumulated knowledge, thereby enhancing its overall decision-making efficiency.

3.2.1 Reflection

Figure 4 illustrates an example of the reflection mechanism applied to a HotpotQA task Yang et al. (2018b). The reflection mechanism equips the assistant A with sparse reward signals, such as binary success/failure states, trajectory $\mathcal{T}_t$, and its stored memory $\mathcal{M}_L$. The assistant processes these inputs, deriving insights from past performance and storing self-reflections $\mathbf{r}_t$ for future decision-making. These self-reflections, richer than scalar rewards, enhance the assistant’s learning capacity and are incorporated into long-term memory:

$$\mathbf{r}_t = \text{ref}(\mathbf{o}_{1:t}, \mathbf{R}_{1:t}), \quad (8)$$

where $\text{ref}(\cdot)$ denotes the reflection function based on the output sequence $\mathbf{o}_{1:t}$ and rewards $\mathbf{R}_{1:t}$. The derived reflection $\mathbf{r}_t$ is then added to $\mathcal{M}_L$:

$$\mathcal{M}_L \leftarrow \mathcal{M}_L \cup \{\mathbf{r}_t\}. \quad (9)$$

The process gradually enhances the agent’s decision-making, allowing it to adapt effectively through accumulated experience.

3.2.2 MemorySyntax

Building upon the reflection mechanism, the MemorySyntax method integrates the Ebbinghaus forgetting curve with linguistic principles to emulate human-like memory processes within the agent’s memory management system. Let I_t denote the information received at time t , and let $R(I_t, \tau)$ represent its retention rate after a time interval τ . According to the Ebbinghaus forgetting curve, the retention rate is modeled as:

$$R(I_t, \tau) = e^{-\frac{\tau}{S}}, \quad (10)$$

where S is the strength of the information, reflecting its importance and complexity.

To enhance retention, MemorySyntax applies linguistic optimization to I_t , producing an optimized version I_t^* with increased strength $S^* > S$. The retention rate for I_t^* is defined as:

$$R(I_t^*, \tau) = \begin{cases} e^{-\frac{\tau}{S^*}}, & \text{if } I_t^* \in \mathcal{M}_S, \\ e^{-\frac{\tau}{S}}, & \text{if } I_t^* \in \mathcal{M}_L, \end{cases} \quad (11)$$

where $\mathcal{M}_S$ and $\mathcal{M}_L$ represent short-term memory and long-term memory, respectively.

The agent updates its memory state $\mathcal{M}_t$ based on the retention rate of I_t^* using predefined thresholds θ_1 and θ_2 , with $\theta_1 > \theta_2$. The memory update rule is formalized as:

$$\mathcal{M}_{t+1} = \begin{cases} \mathcal{M}_t \cup \{I_t^*\}, & \text{if } R(I_t^*, \tau) \geq \theta_1, \\ \mathcal{M}_t \setminus \{I_t^*\}, & \text{if } R(I_t^*, \tau) < \theta_2, \\ \mathcal{M}_t, & \text{otherwise.} \end{cases} \quad (12)$$

This update rule operates under the following conditions:

- **Retention in Short-Term Memory ($\mathcal{M}_S$):** If $R(I_t^*, \tau) \geq \theta_1$, the information I_t^* is retained in short-term memory $\mathcal{M}_S$.
- **Discarding Information:** If $R(I_t^*, \tau) < \theta_2$, the information I_t^* is considered insignificant and is discarded.
- **Transfer to Long-Term Memory ($\mathcal{M}_L$):** If $\theta_2 \leq R(I_t^*, \tau) < \theta_1$, the information I_t^* is transferred to long-term memory $\mathcal{M}_L$.

The above MemorySyntax method is optimal and ensures maximal information utility. For theoretical background and detailed proof, see Appendix A.1.

By simulating human memory dynamics, MemorySyntax enables the agent to prioritize essential information, retain critical data in short-term memory $\mathcal{M}_S$, store important but less frequently used information in long-term memory $\mathcal{M}_L$, and discard irrelevant data. The mechanism addresses memory capacity limitations and enhances the agent’s ability to perform complex tasks requiring efficient memory management.

4 Experiment

To demonstrate the capabilities and performance of SAGE in coordinating autonomous agent groups to work together on tasks, we conduct extensive quantitative experiments on benchmark tasks. We use a public benchmark, AgentBench Liu et al. (2023b), which is a multidimensional evolutionary benchmark from which we select six tasks. These tasks test the reasoning and decision-making abilities of LLMs acting as agents in multi-turn open-ended generation settings. To further assess the agents’ long-context understanding, we select four widely adopted tasks related to long-text processing. These tasks reflect the agents’ programming abilities (LCC Guo et al. (2023), RepoBench-P Liu et al. (2023a)) and reasoning abilities (HotpotQA Yang et al. (2018b), TriviaQA Joshi et al. (2017b)).

4.1 Evaluation on AgentBench

Task Description AgentBench includes scenarios from CODE (Knowledge Graph, OS, DB), GAME (ALFWorld) Shridhar et al. (2021), and WEB (WebShop Yao et al. (2023a), Mind2Web Deng et al. (2023)). For more details for the datasets and benchmarks, see Appendix B.

Baselines We evaluate commercial models GPT-3.5 Brown et al. (2020) and GPT-4 OpenAI et al. (2024), and open-source models Llama2 Touvron et al. (2023), Codellama Rozière et al. (2024), Qwen Bai et al. (2023), and ChatGLM2 GLM et al. (2024). Dialogue history exceeding the model length limit is truncated, using greedy decoding.

Results As shown in Table 1, our method significantly improves model performance on AgentBench, especially for smaller models. GPT-3.5 and GPT-4, despite already high scores, also show notable improvements with SAGE, up to 2.26x in the Database task. Llama2-7b is notably enhanced, showing the method’s impact on weaker models. CodeLlama-7b and Qwen-1.8B also see substantial gains. Qwen-1.8B, after applying our method, performs close to GPT-3.5, highlighting its potential as a general agent. Llama2, previously error-prone, shows a significant reduction in basic errors through feedback and memory optimization, proving that our method not only enhances agent capabilities but also reduces fundamental errors in complex tasks.

Table 1: Baseline and SAGE Framework Performance on AgentBench

LLM Type	Model	VER		OS		DB		KG		ALF		WS		M2W	
		Base	SAGE	Base	SAGE	Base	SAGE	Base	SAGE	Base	SAGE	Base	SAGE	Base	SAGE
API	GPT-4	42.4	49.7	32.0	39.8	57.4	63.1	78.0	82.0	67.1	67.8	27.0	32.0	27.0	32.0
	GPT-3.5	31.6	38.3	15.7	35.6	25.9	37.6	17.0	23.0	64.1	72.1	16.0	28.0	16.0	28.0
OSS	Llama2-7B Chat	0.0	8.4	0.0	10.2	0.0	25.0	0.0	5.0	4.4	10.4	0.0	15.0	0.0	15.0
	CodeLlama-7B Instruct	5.7	18.4	2.6	19.2	0.0	27.0	0.0	12.5	16.3	40.2	0.0	15.0	15.0	15.0
	Qwen1.8B Chat	2.7	18.7	1.4	15.1	6.8	45.3	0.0	10.5	6.6	11.4	0.6	13.6	13.6	13.6
	Qwen-7B Chat	5.6	22.2	4.8	18.0	0.0	48.0	34.0	38.5	0.0	13.6	0.0	15.0	15.0	15.0
	ChatGLM2-6B v1.1	0.0	15.2	0.0	16.3	0.0	17.0	0.0	5.0	0.3	10.3	4.9	14.9	14.9	14.9

4.2 Complex Problem-Solving Tasks Evaluation

We evaluated SAGE against baseline models on three tasks: long-form QA Akash et al. (2023), multi-turn dialog Cui et al. (2020), and sequential task completion Stephens et al. (2012). As shown in Table 2, SAGE outperforms all baselines with significant gains, such as a 20.8% increase in answer accuracy for GPT-3.5 on HotpotQA Yang et al. (2018b) and a 17.3% improvement in task completion for Mistral-7b on ALFWorld Shridhar et al. (2021). Across all tasks, SAGE notably enhances answer accuracy, dialog coherence, and step completion.

4.3 Evaluation of Long-Context Tasks

We evaluated the agent’s code generation and reasoning on four long-text tasks: **LCC Dataset** Mohler et al. (2016) focuses on predicting the next line of code from a few initial lines, with Precision, Recall, and F1 as metrics. **RepoBench-P** Liu et al. (2024) tests retrieval of relevant code snippets from cross-file and within-file contexts to predict the next line, also evaluated with Precision, Recall, and F1. **HotPotQA** Yang et al. (2018a), a Wikipedia-based

Table 2: Evaluation of SAGE and Baseline Models on Three Different Tasks

Agent	Task Completion Time (min)	Answer Accuracy (QA)(%)	Dialog Coherence (%)	Step Completion Accuracy (%)
GPT-3.5 (Baseline)	Long-form QA (HotpotQA)	54.1%	48.5%	62.7%
GPT-4 (Baseline)	Long-form QA (HotpotQA)	61.2%	53.8%	68.2%
Llama2-7b (Baseline)	Multi-turn Dialog (MultiWOZ)	55.9%	50.1%	64.8%
Codellama-13b (Baseline)	Multi-turn Dialog (MultiWOZ)	58.4%	52.3%	66.7%
Mistral-7b (Baseline)	Sequential Task (ALFWorld)	56.5%	51.5%	65.1%
SAGE-GPT-3.5	Long-form QA (HotpotQA)	74.9% (+20.8%)	68.3% (+19.8%)	80.6% (+17.9%)
SAGE-GPT-4	Long-form QA (HotpotQA)	78.4% (+17.2%)	73.4% (+19.6%)	83.9% (+15.7%)
SAGE-Llama2-7b	Multi-turn Dialog (MultiWOZ)	72.2% (+16.1%)	67.9% (+17.8%)	78.5% (+13.7%)
SAGE-Codellama-13b	Multi-turn Dialog (MultiWOZ)	74.7% (+16.3%)	71.2% (+18.9%)	81.2% (+14.5%)
SAGE-Mistral-7b	Sequential Task (ALFWorld)	73.8% (+17.3%)	70.5% (+19.0%)	79.9% (+14.8%)

dataset with 113k question-answer pairs, challenges the agent to reason across multiple documents, evaluated by answer F1. **TriviaQA** Joshi et al. (2017a) is a reading comprehension dataset with question-answer pairs and evidence paragraphs (filtered to over 1,000 words), also using answer F1 for evaluation.

We compared two self-refinement methods: **Beam Search** Kool et al. (2019), which integrates self-assessment through stochastic beam search, and **Reflexion** Shinn et al. (2023), which uses past trial experience in a verbal form.

4.4 Evaluation Results:

Code Completion Task: On the LCC dataset (Table 3), SAGE shows a slight improvement in F1 score (79.29) compared to Beam Search and Reflexion. Its memory mechanisms help refine code predictions, but the performance difference is not substantial in simpler tasks like code completion.

Reasoning Tasks: SAGE significantly outperforms Reflexion and Beam Search on HotPotQA and TriviaQA, with F1 scores of 22.06 and 22.76 (Table 3). SAGE’s ability to effectively integrate multi-document information through reflection leads to better reasoning accuracy, while Reflexion and Beam Search face challenges in handling complex reasoning tasks.

Table 3: Comparison of Performance Across Different Methods

Models	LCC			RepoBench-P			HotpotQA	TriviaQA
	Precision	Recall	F1	Precision	Recall	F1	F1	F1
Reflexion	77.72	81.00	79.28	78.73	81.86	80.25	11.26	11.23
Beam search	78.98	79.32	79.12	78.75	81.02	79.87	10.26	12.13
SAGE	78.76	79.88	79.29	79.27	83.28	81.22	22.06	22.76

4.5 Evaluation of RAG Agents

Table 4 compares classical lexical retrieval (RAG with BM25) Robertson & Zaragoza (2009), dense passage retrieval (RAG with DPR) Reichman & Heck (2024), a commercial retrieval solution (RAG with OpenAI Retrieval) OpenAI (2023), the TART Eisenschlos et al. (2022) sequence-to-sequence retrieval model, and the FiD (Fusion-in-Decoder) method Izacard & Grave (2021), all tested on multi-document and open-domain QA tasks (HotpotQA, Natural Questions Kwiatkowski et al. (2019), and TriviaQA). RAG with BM25 relies on term-based matching, while RAG with DPR uses learned dense embeddings. TART adopts a transformer-based approach to produce relevant contexts, and FiD fuses multiple retrieved passages through an encoder-decoder design. In contrast, ChatGPT-4 (SAGE) employs a structured reasoning workflow for retrieval and generation, which leads to steady accuracy improvements of 3.6% to 4.7% and cuts memory consumption nearly 50% on some tasks, all without increasing latency.

Table 4: Evaluation of different RAG Agents on Different Tasks and Datasets

Agent	Accuracy (QA) (%)	Latency (ms)	Memory Usage (MB)
Task 1: Multi-Document QA (HotpotQA)			
RAG (BM25)	60.8	121	613
RAG (DPR)	66.3	129	542
RAG (OpenAI Retrieval)	67.4	108	494
TART	63.2	144	477
FiD (Fusion-in-Decoder)	70.1	153	456
ChatGPT-4 - Sage	74.8 (+4.7)	128	231 (-50%)
Task 2: Document Retrieval for Contextual Answering (Natural Questions)			
RAG (BM25)	59.9	125	605
RAG (DPR)	65.5	131	561
RAG (OpenAI Retrieval)	66.8	113	484
TART	62.4	146	455
FiD (Fusion-in-Decoder)	69.8	156	443
ChatGPT-4 - Sage	73.6 (+3.8)	131	227 (-49%)
Task 3: Open-Domain QA with Multiple Contexts (TriviaQA)			
RAG (BM25)	62.1	124	615
RAG (DPR)	67.8	129	530
RAG (OpenAI Retrieval)	68.9	117	494
TART	64.7	148	462
FiD (Fusion-in-Decoder)	71.9	155	456
ChatGPT-4 - Sage	75.5 (+3.6)	134	243 (-47%)

4.6 Error analysis

As shown in Figure 3, the SAGE framework significantly enhances agent performance across tasks, especially in the WS task for AgentBench, due to its iterative feedback mechanism, which refines outputs through continuous assistant-checker interaction. In OS and DB tasks, Context Limit Exceeded and invalid format errors are nearly eliminated, with a notable reduction in invalid action errors, attributed to the reflection mechanism that helps the assistant learn and reduce logical mistakes.

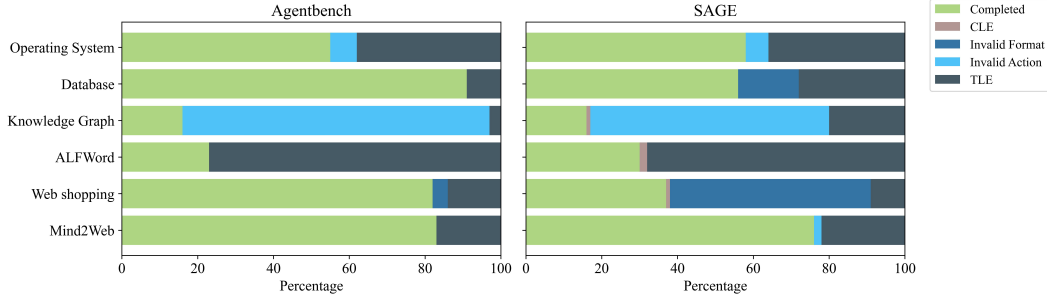


Figure 3: Execution results across six tasks (CLE: Context Limit Exceeded, TLE: Task Limit Exceeded). Task limits are the main cause of incomplete tasks, highlighting LLM agents' limitations under time constraints.

4.7 Ablation Study

We conducted ablation experiments on Qwen-1.8B and CodeLlama-7B to evaluate memory optimization (Table 5). Without memory optimization, both models perform weakly, especially Qwen-1.8B, which improves from 6.8 to 48.0 in KG and from 0.0 to 10.5 in ALF after optimization. Similarly, CodeLlama-7B shows substantial gains, particularly in DB (2.7 to 41.3) and WS (14.3 to 58.7). Overall, CodeLlama-7B performs better than Qwen-1.8B,

highlighting the stronger adaptability of models with more parameters in handling complex tasks.

Table 5: Ablation study for memory optimization on the task of AgentBench

Models	OS	DB	KG	ALF	WS	M2W
Qwen-1.8B (w/o memo)	10.4	22.6	6.8	0.0	26.6	5.0
Qwen-1.8B (w memo)	18.7	28.3	45.3	10.5	31.4	25.1
Codellama-7B (w/o memo)	9.7	2.7	0.0	0.0	14.3	5.0
Codellama-7B (w memo)	23.4	41.3	48.0	12.5	58.7	15.0

5 Conclusion

In this paper, we propose the SAGE framework, which enhances agents’ self-adjustment and memory management in complex tasks through reflective mechanisms and memory optimization. Experimental results show significant performance improvements across benchmarks, especially in smaller models. In the AgentBench test, SAGE boosts the performance of strong baselines like GPT-3.5 and GPT-4, while also significantly improving open-source models. It effectively reduces basic errors and logical mistakes, particularly enabling smaller models to handle complex tasks.

References

- Pritom Saha Akash, Kashob Kumar Roy, Lucian Popa, and Kevin Chen-Chuan Chang. Long-form question answering: An iterative planning-retrieval-generation approach, 2023. URL <https://arxiv.org/abs/2311.09383>.
- Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Sheng-guang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. Qwen technical report, 2023.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020.
- Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory (Wiley Series in Telecommunications and Signal Processing)*. Wiley-Interscience, USA, 2006. ISBN 0471241954.
- Leyang Cui, Yu Wu, Shujie Liu, Yue Zhang, and Ming Zhou. MuTual: A dataset for multi-turn dialogue reasoning. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 1406–1416, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.130. URL <https://aclanthology.org/2020.acl-main.130/>.
- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web, 2023.
- Hermann Ebbinghaus. *Über das gedächtnis: untersuchungen zur experimentellen psychologie*. Duncker & Humblot, 1885.
- Julian Eisenschlos, Yury Zemlyanskiy, Jason Phang, Bhuwan Dhingra, and William W. Cohen. TART: Retrieval-augmented generation for task-oriented dialog. In *Proceedings of*

- the 60th Annual Meeting of the Association for Computational Linguistics (ACL), pp. 1866–1881, 2022. URL <https://aclanthology.org/2022.acl-long.131>.
- Romina Etezadi and Mehrnoush Shamsfard. The state of the art in open domain complex question answering: a survey. *Applied Intelligence*, 53(4):4124–4144, 2023.
- Drew Fudenberg and Jean Tirole. *Game Theory*. MIT Press, Cambridge, MA, 1991.
- Team GLM, Aohan Zeng, Bin Xu, Bowen Wang, Chenhui Zhang, Da Yin, Diego Rojas, Guanyu Feng, Hanlin Zhao, Hanyu Lai, Hao Yu, Hongning Wang, Jiadai Sun, Jiajie Zhang, Jiale Cheng, Jiayi Gui, Jie Tang, Jing Zhang, Juanzi Li, Lei Zhao, Lindong Wu, Lucen Zhong, Mingdao Liu, Minlie Huang, Peng Zhang, Qinkai Zheng, Rui Lu, Shuaiqi Duan, Shudan Zhang, Shulin Cao, Shuxun Yang, Weng Lam Tam, Wenyi Zhao, Xiao Liu, Xiao Xia, Xiaohan Zhang, Xiaotao Gu, Xin Lv, Xinghan Liu, Xinyi Liu, Xinyue Yang, Xixuan Song, Xunkai Zhang, Yifan An, Yifan Xu, Yilin Niu, Yuantao Yang, Yueyan Li, Yushi Bai, Yuxiao Dong, Zehan Qi, Zhaoyu Wang, Zhen Yang, Zhengxiao Du, Zhenyu Hou, and Zihan Wang. Chatglm: A family of large language models from glm-130b to glm-4 all tools, 2024.
- Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, et al. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476, 2016.
- Daya Guo, Canwen Xu, Nan Duan, Jian Yin, and Julian McAuley. Longcoder: A long-range pre-trained language model for code completion, 2023.
- Yangfan He, Jianhui Wang, Kun Li, Yijin Wang, Li Sun, Jun Yin, Miao Zhang, and Xueqian Wang. Enhancing intent understanding for ambiguous prompt: A human-machine co-adaptation strategy. *Available at SSRN 5119629*, 2024.
- Yangfan He, Jianhui Wang, Kun Li, Yijin Wang, Li Sun, Jun Yin, Miao Zhang, and Xueqian Wang. Enhancing intent understanding for ambiguous prompts through human-machine co-adaptation. *arXiv preprint arXiv:2501.15167*, 2025.
- Jiaxin Huang, Shixiang Shane Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. Large language models can self-improve, 2022.
- Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume (EACL)*, pp. 874–880, 2021. URL <https://aclanthology.org/2021.eacl-main.24>.
- Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In Regina Barzilay and Min-Yen Kan (eds.), *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1601–1611, Vancouver, Canada, July 2017a. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL <https://aclanthology.org/P17-1147>.
- Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension, 2017b.
- Wouter Kool, Herke van Hoof, and Max Welling. Stochastic beams and where to find them: The gumbel-top-k trick for sampling sequences without replacement, 2019. URL <https://arxiv.org/abs/1903.06059>.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. doi: 10.1162/tacl_a_00276. URL <https://aclanthology.org/Q19-1026/>.

- Zeyuan Li, Yangfan He, Lewei He, Jianhui Wang, Tianyu Shi, Bin Lei, Yuchen Li, and Qiuwu Chen. Falcon: Feedback-driven adaptive long/short-term memory reinforced coding optimization system. *arXiv preprint arXiv:2410.21349*, 2024.
- Xuechen Liang, Meiling Tao, Yinghui Xia, Tianyu Shi, Jun Wang, and JingSong Yang. Cmat: A multi-agent collaboration tuning framework for enhancing small language models. *arXiv preprint arXiv:2404.01663*, 2024.
- Tianyang Liu, Canwen Xu, and Julian McAuley. Repobench: Benchmarking repository-level code auto-completion systems, 2023a.
- Tianyang Liu, Canwen Xu, and Julian McAuley. Repobench: Benchmarking repository-level code auto-completion systems. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=pPjZIOuQuF>.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023b.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.
- Michael Mohler, Mary Brunson, Bryan Rink, and Marc Tomlinson. Introducing the LCC metaphor datasets. In Nicoletta Calzolari, Khalid Choukri, Thierry Declerck, Sara Goggi, Marko Grobelnik, Bente Maegaard, Joseph Mariani, Helene Mazo, Asuncion Moreno, Jan Odijk, and Stelios Piperidis (eds.), *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC’16)*, pp. 4221–4227, Portorož, Slovenia, May 2016. European Language Resources Association (ELRA). URL <https://aclanthology.org/L16-1668>.
- Yohei Nakajima. Babyagi: An ai agent that can achieve goals and execute tasks, 2024.
- OpenAI. Openai cookbook. <https://github.com/openai/openai-cookbook>, 2023. Accessed: 2025-03-27.
- OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Floren-
cia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat,
Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao,
Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro,
Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brak-
man, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie
Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke
Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen,
Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings,
Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien
Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty
Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman,
Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun
Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott
Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han,
Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade
Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost
Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun
Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali
Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick,
Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt
Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis,
Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike,

- Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rameev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O’Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. Gpt-4 technical report, 2024. URL <https://arxiv.org/abs/2303.08774>.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems, 2024.
- Xiangfei Qiu, Jilin Hu, Lekui Zhou, Xingjian Wu, Junyang Du, Buang Zhang, Chenjuan Guo, Aoying Zhou, Christian S. Jensen, Zhenli Sheng, and Bin Yang. Tfb: Towards comprehensive and fair benchmarking of time series forecasting methods. In *Proc. VLDB Endow.*, pp. 2363–2377, 2024.
- Xiangfei Qiu, Xiuwen Li, Ruiyang Pang, Zhicheng Pan, Xingjian Wu, Liu Yang, Jilin Hu, Yang Shu, Xuesong Lu, Chengcheng Yang, Chenjuan Guo, Aoying Zhou, Christian S. Jensen, and Bin Yang. Easytime: Time series forecasting made easy. In *ICDE*, 2025a.
- Xiangfei Qiu, Xingjian Wu, Yan Lin, Chenjuan Guo, Jilin Hu, and Bin Yang. Duet: Dual clustering enhanced multivariate time series forecasting. In *SIGKDD*, pp. 1185–1196, 2025b.
- Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, and Timothy P Lillicrap. Compressive transformers for long-range sequence modelling. *arXiv preprint arXiv:1911.05507*, 2019.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.
- Benjamin Reichman and Larry Heck. Dense passage retrieval: Is it retrieving?, 2024. URL <https://arxiv.org/abs/2402.11035>.
- Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389, 2009. ISSN 1554-0669. doi: 10.1561/1500000019. URL <http://dx.doi.org/10.1561/1500000019>.
- Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori,

- Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. Code llama: Open foundation models for code, 2024. URL <https://arxiv.org/abs/2308.12950>.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023.
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021. URL <https://arxiv.org/abs/2010.03768>.
- Keri K Stephens, Jaehee K Cho, and Dawna I Ballard. Simultaneity, sequentiality, and speed: Organizational messages about multiple-task completion. *Human Communication Research*, 38(1):23–47, 2012.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.
- Ye Tian, Baolin Peng, Linfeng Song, Lifeng Jin, Dian Yu, Haitao Mi, and Dong Yu. Toward self-improvement of llms via imagination, searching, and criticizing, 2024.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.
- Takashi Wada, Tomoharu Iwata, and Yuji Matsumoto. Unsupervised multilingual word embedding with limited resources using neural language models. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 3113–3124, 2019.
- Hui Yang, Sifu Yue, and Yunzhong He. Auto-gpt for online decision making: Benchmarks and additional opinions, 2023. URL <https://arxiv.org/abs/2306.02224>.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun’ichi Tsujii (eds.), *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 2369–2380, Brussels, Belgium, October–November 2018a. Association for Computational Linguistics. doi: 10.18653/v1/D18-1259. URL <https://aclanthology.org/D18-1259>.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering, 2018b. URL <https://arxiv.org/abs/1809.09600>.
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents, 2023a.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023b. URL <https://arxiv.org/abs/2210.03629>.

Ziqi Zhou, Jingyue Zhang, Jingyuan Zhang, Yangfan He, Boyue Wang, Tianyu Shi, and Alaa Khamis. Human-centric reward optimization for reinforcement learning-based automated driving using large language models. *arXiv preprint arXiv:2405.04135*, 2024.

User: Answer the question based on the given passages. Only give me the answer and do not output any other words. The following are given passages:
The "trustly system" (sometimes incorrectly called "trustee system") was a penitentiary system of discipline and security enforced in parts of the United States until the 1980s
Answer the question based on the given passages. Only give me the answer and do not output any other words.
Question: Which case was brought to court first Miller v. California or Gates v. Collier ?
Answer:

Assistant: Gates v. Collier

Checker:
Incorrect answer: Neglecting specific years: Assistant did not accurately confirm the specific years of each case, leading to an incorrect judgment of the chronological order.
Insufficient understanding of the question: When answering the question, Assistant may not have fully understood or carefully analyzed the provided information, resulting in a misjudgment.

Reflection:
When answering the question about which case, "Miller v. California" or "Gates v. Collier," was brought up first, I mistakenly thought "Gates v. Collier" preceded "Miller v. California." In reality, "Miller v. California" came first. This indicates that I made an error in judging the chronological order of events.
Improvement measures: Carefully verify information: When answering questions involving time sequences, it is essential to carefully check the specific timing of each event to ensure accuracy.
Timeline tools: When dealing with multiple events or cases, use timeline tools to help clarify the chronological order and avoid oversights.
Review mechanism: Before submitting the final answer, establish a self-review mechanism to recheck all relevant information and details to ensure the accuracy of the response.
Conclusion: This mistake serves as a reminder for me to be more careful and cautious when handling questions related to time and sequence. Through reflection and improvement, I hope to provide more accurate and reliable answers in the future and prevent similar errors from occurring.

Assistant with SAGE response:
Miller v. California

Figure 4: The illustration of an example HotpotQA with SAGE.

A Theoretical Analysis

A.1 An Information-Theoretic Perspective on Memory Optimization Based on the Ebbinghaus Forgetting Curve

The memory optimization mechanism proposed in this work is grounded in the Ebbinghaus forgetting curve. It dynamically manages short-term memory (STM) and long-term memory (LTM) by prioritizing the retention of essential information while discarding irrelevant data. This mechanism can be viewed as computing the retention strength $S(I_t)$ of each information unit based on its information entropy. Theoretically, one can interpret this as maximizing information entropy to show that the mechanism effectively mitigates the issue of information overload [Cover & Thomas \(2006\)](#). That is, by preferentially retaining high-entropy (i.e., high-uncertainty) information, the model improves its information processing efficiency.

First, the retention strength of information is defined as:

$$S(I_t) = \frac{H(I_t)}{f(t)} \quad (13)$$

where the numerator $H(I_t)$ represents the information entropy of the content at time t , and $f(t)$ models the forgetting curve.

The objective of memory optimization is to maximize the utility of retained information under a memory capacity constraint:

$$\max \sum_t M_t \cdot S(I_t), \quad \text{s.t.} \quad \sum_t M_t \leq C \quad (14)$$

Here, C represents the total memory capacity, and $M_t \in \{0, 1\}$ denotes whether the information at time t is retained (1) or discarded (0).

Using the method of Lagrange multipliers, we construct the Lagrangian function:

$$\mathcal{L}(M_t, \lambda) = \sum_t M_t \cdot S(I_t) - \lambda \left(\sum_t M_t - C \right) \quad (15)$$

Taking the derivative of $\mathcal{L}$, which is the optimal memory retention, with respect to M_t and setting it to zero yields the optimality condition:

$$\frac{\partial \mathcal{L}}{\partial M_t} = S(I_t) - \lambda = 0 \Rightarrow \begin{cases} \text{Retain } I_t & \text{if } S(I_t) \geq \lambda \\ \text{Discard } I_t & \text{if } S(I_t) < \lambda \end{cases} \quad (16)$$

This rule is consistent with the threshold-based memory retention strategy used in the SAGE framework.

A.2 Iterative Feedback and Game-Theoretic Model

Multi-agent systems often depend on reinforcement learning (RL) methods, making their theoretical basis essential for ensuring that algorithms converge to stable solutions, such as Nash equilibria [Fudenberg & Tirole \(1991\)](#). The subsequent theory provides a formal guarantee of convergence, showing that our proposed algorithm is not randomly designed. Instead, it surpasses standard approaches by establishing robust criteria that confirm both its stability and performance advantages.

A.2.1 Players and Their Strategy Spaces Settings

- User (U)** : Provides a task description T_U and constraints C_U .
- Assistant (A)** : Chooses actions $a_t \sim \pi_\theta(\cdot | s_t)$, where π_θ is the policy. (17)
- Checker (C)** : Generates feedback f_t aimed at reducing the error $\|o_t - o^*\|_F$,

where o_t is the output of the assistant and o^* is the expected (ground-truth) output, $\|\cdot\|_F$ denotes the Frobenius norm.

For the Utility Functions, we have:

- Assistant's Utility, R_A** : Measures task performance (e.g., BLEU score, accuracy):
 $R_A = \text{Task Performance Metric of } a_t$.
 - Checker's Utility, R_C** : Measures feedback accuracy by reduction of error:
 $R_C \propto \Delta \|o_t - o^*\|_F$.
- (18)

The overall interaction can be viewed as a three-player game in which: The assistant (A) optimizes its policy π_θ to maximize R_A , while the checker (C) aims to minimize $\|o_t - o^*\|$ (i.e., maximize R_C). The user U sets the task and constraints that guide both the assistant and checker's performances. Next we want to show that it can be shown that under a finite strategy space and continuous utility functions, a Nash equilibrium exists in this multi-agent game.

A.2.2 Setup for the proof of theorem 3.1

First, we want to write a road map for this proof.

- It can be shown that under a finite strategy space and continuous utility functions, a Nash equilibrium exists in this multi-agent game.
- According to the **Debreu-Glicksberg-Fan Theorem** (a generalization of Nash's existence theorem), if each player's strategy space is a compact convex set and the utility functions are continuous and quasi-concave (or concave) in their own strategies, then at least one Nash equilibrium exists.
- Then, the assistant's policy π_θ is updated using policy gradient methods. Since R_A is a continuous function of π_θ , the assistant's utility is continuous.

- After that, the checker's feedback strategy can be obtained via convex optimization (e.g., least-squares regression), and thus R_C is a convex function.
- Therefore, a Nash equilibrium (π_θ^*, f^*) exists, at which both the assistant and checker's strategies are stable with respect to each other:

$$\pi_\theta^* = \arg \max_{\pi} R_A(\pi, f^*), \quad f^* = \arg \max_f R_C(\pi_\theta^*, f) \quad (19)$$

Let's start the proof.

Lemma A.1. *Let $\mathcal{S}_1, \mathcal{S}_2, \dots, \mathcal{S}_N$ be finite strategy sets for each of N players, and let $u_i : \mathcal{S}_1 \times \dots \times \mathcal{S}_N \rightarrow \mathbb{R}$ be continuous for each i . Then, there exists a Nash equilibrium in the multi-agent game.*

Proof. For each player i , define the set of mixed strategies as

$$\Delta(\mathcal{S}_i) = \left\{ x_i \in \mathbb{R}^{|\mathcal{S}_i|} \mid x_i(s) \geq 0 \text{ for all } s \in \mathcal{S}_i, \sum_{s \in \mathcal{S}_i} x_i(s) = 1 \right\}. \quad (20)$$

Each set $\Delta(\mathcal{S}_i)$ is a simplex in $\mathbb{R}^{|\mathcal{S}_i|}$, and hence is nonempty, compact, and convex. Define the expected utility function for player i :

$$U_i(x_1, \dots, x_N) = \sum_{(s_1, \dots, s_N) \in \mathcal{S}_1 \times \dots \times \mathcal{S}_N} \left(\prod_{j=1}^N x_j(s_j) \right) u_i(s_1, \dots, s_N). \quad (21)$$

Since the strategy spaces are finite and u_i is continuous (and hence bounded on the finite set), the function U_i is continuous on the product space $\prod_{j=1}^N \Delta(\mathcal{S}_j)$.

For each player i , define the best response correspondence as:

$$BR_i(x_{-i}) = \arg \max_{x_i \in \Delta(\mathcal{S}_i)} U_i(x_i, x_{-i}), \quad (22)$$

where x_{-i} denotes the mixed strategies of all players other than i . Because $\Delta(\mathcal{S}_i)$ is compact and U_i is continuous and linear in x_i (hence concave), the maximum exists and $BR_i(x_{-i})$ is nonempty, convex, and compact. Now we define the overall best-response correspondence:

$$BR(x) = BR_1(x_{-1}) \times BR_2(x_{-2}) \times \dots \times BR_N(x_{-N}), \quad (23)$$

where $x = (x_1, x_2, \dots, x_N)$. The product space $\prod_{i=1}^N \Delta(\mathcal{S}_i)$ is compact and convex, and the correspondence BR has nonempty, convex, and compact values and is upper hemicontinuous (due to the continuity and linearity of the expected utilities). Hence, by Kakutani's fixed point theorem, there exists a fixed point

$$x^* = (x_1^*, x_2^*, \dots, x_N^*) \in \prod_{i=1}^N \Delta(\mathcal{S}_i) \quad (24)$$

such that

$$x^* \in BR(x^*). \quad (25)$$

This fixed point is a Nash equilibrium in mixed strategies.

Thus, under a finite strategy space and continuous utility functions, a Nash equilibrium exists. According to the **Debreu-Glicksberg-Fan Theorem** (a generalization of Nash's existence theorem), if each player's strategy space is a compact convex set and the utility functions are continuous and quasi-concave (or concave) in their own strategies, then at least one Nash equilibrium exists. □

Lemma A.2. *By Lemma A.1, Let $I = \{1, 2, \dots, n\}$ be the set of players. For each player i , let $X_i \subset \mathbb{R}^{d_i}$ be a compact, convex strategy set, and define $X = \prod_{i=1}^n X_i$. Suppose each player's utility function $u_i : X \rightarrow \mathbb{R}$ is continuous on X and quasi-concave (or concave) in the coordinate x_i for fixed x_{-i} . Then, by the Debreu-Glicksberg-Fan theorem, there exists at least one Nash equilibrium $x^* = (x_1^*, x_2^*, \dots, x_n^*) \in X$.*

Proof. For each player i , define the best-response correspondence

$$BR_i(x_{-i}) = \{x_i \in X_i : u_i(x_i, x_{-i}) = \max_{y_i \in X_i} u_i(y_i, x_{-i})\}. \quad (26)$$

Since u_i is continuous and X_i is compact, the maximum is attained; hence, $BR_i(x_{-i}) \neq \emptyset$. The quasi-concavity (or concavity) of u_i in x_i ensures that the set

$$\{x_i \in X_i : u_i(x_i, x_{-i}) \geq \alpha\} \quad (27)$$

is convex for any $\alpha \in \mathbb{R}$. In particular, the set $BR_i(x_{-i})$ is convex for every fixed x_{-i} .

The continuity of u_i implies that the best-response correspondence BR_i is upper hemicontinuous. This means that for any sequence $\{x_{-i}^{(k)}\}$ converging to x_{-i} and any corresponding sequence $\{x_i^{(k)}\}$ with $x_i^{(k)} \in BR_i(x_{-i}^{(k)})$, every limit point x_i of $\{x_i^{(k)}\}$ belongs to $BR_i(x_{-i})$.

Now we define the aggregate best-response correspondence:

$$BR(x) = BR_1(x_{-1}) \times BR_2(x_{-2}) \times \cdots \times BR_n(x_{-n}), \quad (28)$$

where $x = (x_1, x_2, \dots, x_n) \in X$. Note that: (1) X is a nonempty, compact, convex subset of $\mathbb{R}^{d_1+d_2+\dots+d_n}$. (2) For each i , $BR_i(x_{-i})$ is nonempty and convex. (3) The correspondence BR is upper hemicontinuous.

By the Debreu-Glicksberg-Fan fixed point theorem, there exists a fixed point $x^* \in X$ such that $x^* \in BR(x^*)$. That is, for every $i \in I$, $x_i^* \in BR_i(x_{-i}^*)$. This means that no player can unilaterally deviate from x^* and increase their utility, which is precisely the definition of a Nash equilibrium. □

Next, based on the above Lemmas, we want to prove that the assistant's policy π_θ is updated using policy gradient methods Sutton & Barto (2018). Since R_A is a continuous function of π_θ , the assistant's utility is continuous.

Lemma A.3. Let Θ denote the parameter space for the assistant's policy π_θ , and let the assistant's utility be given by

$$R_A(\pi_\theta) = f(\pi_\theta), \quad (29)$$

where $f : \Pi \rightarrow \mathbb{R}$ is a continuous function on the space Π of policies. Assume that the mapping $\theta \mapsto \pi_\theta$ is continuous. Then the composite function

$$F(\theta) = f(\pi_\theta) \quad (30)$$

is continuous in θ . Moreover, since the assistant's policy is updated using policy gradient methods, the update rule

$$\theta_{k+1} = \theta_k + \alpha_k \nabla_\theta F(\theta_k) \quad (31)$$

employs the gradient of a continuous function.

Proof. We divide the proof into two parts:

Step 1: Continuity of the Utility Function. Since the mapping $\theta \mapsto \pi_\theta$ is continuous and the function $f : \Pi \rightarrow \mathbb{R}$ is continuous by hypothesis, their composition $F(\theta) = f(\pi_\theta)$ is continuous in θ . Formally, for any $\epsilon > 0$, there exists a $\delta > 0$ such that for all $\theta, \theta' \in \Theta$ satisfying $\|\theta - \theta'\| < \delta$,

$$|F(\theta) - F(\theta')| = |f(\pi_\theta) - f(\pi_{\theta'})| < \epsilon. \quad (32)$$

Step 2: Policy Gradient Updates. In the policy gradient framework, the assistant updates its parameters according to the rule:

$$\theta_{k+1} = \theta_k + \alpha_k \nabla_\theta F(\theta_k), \quad (33)$$

where $\alpha_k > 0$ is a (possibly diminishing) step size and $\nabla_\theta F(\theta_k)$ denotes the gradient of F evaluated at θ_k . Since F is continuous (and under typical smoothness assumptions,

differentiable with continuous gradient), the gradient $\nabla_\theta F$ is well-defined. The policy gradient update is thus a continuous map from θ_k to θ_{k+1} .

Because the update at each iteration is based on the gradient (which is continuous) of the assistant's utility $F(\theta)$, any small perturbation in θ results in only a small variation in both the gradient and the updated parameter. This demonstrates that the update procedure is stable in the sense that the assistant's utility, $R_A(\pi_\theta) = F(\theta)$, continuously varies under the sequence of policy gradient updates [Sutton & Barto \(2018\)](#).

Thus, since R_A is a continuous function of π_θ and π_θ depends continuously on θ , the assistant's utility is continuous, and the policy gradient update is well-defined. $\square$

Then we want to prove that, the checker's feedback strategy can be obtained via convex optimization (e.g., least-squares regression), and thus R_C is a convex function.

Lemma A.4. *Let $\mathcal{F}$ be a convex subset of a vector space, and suppose the checker's feedback strategy is given by a function $f \in \mathcal{F}$ obtained as a solution to the convex optimization problem*

$$\min_{f \in \mathcal{F}} J(f) = \sum_{i=1}^n (y_i - f(x_i))^2, \quad (34)$$

where x_i are given inputs and y_i are target outputs. Define the checker's utility function

$$R_C(f) = J(f). \quad (35)$$

Then R_C is a convex function on $\mathcal{F}$; that is, for any $f_1, f_2 \in \mathcal{F}$ and any $\lambda \in [0, 1]$ it holds that

$$R_C(\lambda f_1 + (1 - \lambda)f_2) \leq \lambda R_C(f_1) + (1 - \lambda)R_C(f_2). \quad (36)$$

Proof. We prove the convexity of R_C by verifying that the squared-error loss is a convex function and that convex combinations preserve convexity.

For each data point i , consider the function

$$\ell_i(f) = (y_i - f(x_i))^2. \quad (37)$$

Since for any real number t , the function t^2 is convex, and the mapping $f \mapsto y_i - f(x_i)$ is affine (and hence convex), the composition $f \mapsto (y_i - f(x_i))^2$ is convex. In particular, for any $f_1, f_2 \in \mathcal{F}$ and $\lambda \in [0, 1]$,

$$\ell_i(\lambda f_1 + (1 - \lambda)f_2) \leq \lambda \ell_i(f_1) + (1 - \lambda)\ell_i(f_2). \quad (38)$$

Since the checker's overall loss (and hence utility function)

$$R_C(f) = \sum_{i=1}^n \ell_i(f) \quad (39)$$

is a finite sum of convex functions, it follows that R_C is convex on $\mathcal{F}$; that is, for any $f_1, f_2 \in \mathcal{F}$ and $\lambda \in [0, 1]$,

$$\begin{aligned} R_C(\lambda f_1 + (1 - \lambda)f_2) &= \sum_{i=1}^n \ell_i(\lambda f_1 + (1 - \lambda)f_2) \\ &\leq \lambda \sum_{i=1}^n \ell_i(f_1) + (1 - \lambda) \sum_{i=1}^n \ell_i(f_2) \\ &= \lambda R_C(f_1) + (1 - \lambda)R_C(f_2). \end{aligned} \quad (40)$$

Thus:

$$R_C(f) = \sum_{i=1}^n (y_i - f(x_i))^2 \text{ is convex on } \mathcal{F}. \quad (41)$$

$\square$

Thus, the checker's feedback strategy derived via convex optimization (e.g., least-squares regression) ensures that the corresponding utility function R_C is convex.

A.2.3 The formal proof of theorem 3.1

We want to rewrite the theorem 3.1 into the following lemma based on the former lemmas:

Lemma A.5 (theorem 3.1). *Suppose the assistant’s strategy space Π and the checker’s strategy space $\mathcal{F}$ are nonempty, compact, and convex sets. Assume that: $R_A : \Pi \times \mathcal{F} \rightarrow \mathbb{R}$ is continuous in π for fixed f , $R_C : \Pi \times \mathcal{F} \rightarrow \mathbb{R}$ is convex in f for fixed π . Then there exists a Nash equilibrium π_θ^*, f^* defined by*

$$\pi_\theta^* = \arg \max_{\pi \in \Pi} R_A(\pi, f^*) \quad \text{and} \quad f^* = \arg \max_{f \in \mathcal{F}} R_C(\pi_\theta^*, f). \quad (42)$$

Proof. For each fixed $f \in \mathcal{F}$ define the assistant’s best-response set:

$$BR_A(f) = \{\pi \in \Pi : R_A(\pi, f) = \max_{\pi' \in \Pi} R_A(\pi', f)\}. \quad (43)$$

By hypothesis (i) and the compactness of Π , the maximum is attained and $BR_A(f)$ is nonempty. (Continuity of R_A assures that small changes in f lead to small changes in $R_A(\cdot, f)$.)

Similarly, for each fixed $\pi \in \Pi$ define the checker’s best-response set:

$$BR_C(\pi) = \{f \in \mathcal{F} : R_C(\pi, f) = \max_{f' \in \mathcal{F}} R_C(\pi, f')\}. \quad (44)$$

Since hypothesis (ii) implies that R_C is convex in f and $\mathcal{F}$ is convex and compact, we conclude that $BR_C(\pi)$ is nonempty, convex, and closed.

Consider the correspondence

$$H : \Pi \times \mathcal{F} \rightarrow \Pi \times \mathcal{F}, \quad H(\pi, f) = BR_A(f) \times BR_C(\pi). \quad (45)$$

By the arguments above: For every $(\pi, f) \in \Pi \times \mathcal{F}$, the set $H(\pi, f)$ is nonempty. $BR_A(f)$ is closed (by continuity and compactness) and $BR_C(\pi)$ is convex (and closed) by hypothesis (ii). Hence, $H(\pi, f)$ is convex and closed.

Since the domain $\Pi \times \mathcal{F}$ is nonempty, compact, and convex, and since the correspondence H has nonempty, convex, and closed values and is upper hemicontinuous (this follows from the continuity of R_A in π and convexity of R_C in f), Kakutani’s fixed point theorem guarantees the existence of a fixed point

$$(\pi^*, f^*) \in \Pi \times \mathcal{F} \quad \text{such that} \quad (\pi^*, f^*) \in H(\pi^*, f^*). \quad (46)$$

By the definition of H , the fixed point satisfies

$$\pi^* \in BR_A(f^*) \quad \text{and} \quad f^* \in BR_C(\pi^*), \quad (47)$$

which is equivalent to

$$\pi^* = \arg \max_{\pi \in \Pi} R_A(\pi, f^*) \quad \text{and} \quad f^* = \arg \max_{f \in \mathcal{F}} R_C(\pi^*, f). \quad (48)$$

Thus, no unilateral deviation by either the assistant or the checker increases the respective utility; in other words, the strategy pair (π^*, f^*) is a Nash equilibrium. $\square$

B Detailed Dataset Information

- (1) **Operating systems (OS)** Integrating LLMs into operating systems has immense potential for automating and optimizing tasks. This integration requires a secure, user-friendly interface to ensure effective LLM-OS interaction. LLMs must accurately understand the OS context for informed operations, prioritizing safety to prevent misuse. Additionally, the system should effectively handle errors and provide clear feedback to users, enhancing overall interaction and control. Addressing these aspects can transform computer interaction and efficiency across various industries.

- (2) **Database (DB)** The ability of LLMs to operate on real databases via SQL is critical due to the importance and complexity of database analysis in everyday activities. Previous research has highlighted the effectiveness of LLMs in automating database access, such as with T5QL, a new SQL generation method. Furthermore, fine-tuned LLMs (like GPT-3.5) have demonstrated the ability to extract and link complex scientific information from texts, obtaining structured knowledge from unstructured text and building extensive databases.
- (3) **WebShop (WS)** WebShop is an innovative simulation of an e-commerce website environment, featuring 1.18 million real-world products and 12,087 crowd-sourced text instructions. It challenges agents to navigate various types of webpages and perform diverse actions to find, customize, and purchase products based on given instructions. WebShop’s challenges include understanding compositional instructions, query (re-)formulation, dealing with noisy text on webpages, and strategic exploration.
- (4) **Knowledge Graphs (KG)** LLMs’ utilization in constructing and interacting with knowledge graphs (KG) presents a promising opportunity to enhance semantic understanding and information retrieval. This involves assessing the models’ ability to generate and interpret complex interrelations within data, facilitating more intuitive and context-aware responses. Effective LLM performance in this domain could significantly improve AI’s reasoning and decision-making capabilities based on structured knowledge.
- (5) **Mind2Web (M2W)** Mind2Web (M2W) is a dataset designed for developing web agents capable of performing complex tasks on real websites via language instructions. It features over 2,000 tasks across 137 sites from 31 domains. M2W’s real web environments and diverse user interactions make it a crucial platform for advancing AI navigation capabilities.
- (6) **ALFWorld (ALF)** ALFWorld bridges interactive TextWorld environments with embodied tasks from the ALFRED dataset, enabling agents to learn abstract strategies and apply them to real-world tasks. It supports abstract reasoning and concrete execution, allowing agents to plan actions in a text-based simulator and execute these tasks in a visual environment. This approach enhances agent generalization and problem-solving skills across various domains, such as language understanding and visual navigation, by leveraging a modular design that simplifies research improvements.

C Comparison with Task-Splitting Agents

To compare the SAGE framework with existing agents across various task types and datasets, we conducted a comprehensive evaluation. The results are detailed in Table 6. SAGE demonstrates superior performance in task completion time, answer accuracy, dialog coherence, and step completion accuracy, highlighting its effectiveness in managing complex, multi-faceted tasks.

Table 6: Evaluation of Task-Splitting SOTA Agents on Different Task Types and Datasets

Agent	Task Completion Time (min)	Answer Accuracy (QA)(%)	Dialog Coherence (%)	Step Completion Accuracy (%)
Task 1: Long-form Question Answering (HotpotQA)				
BabyAGI	7.8	74.1	–	–
AutoGPT	8.1	76.3	–	–
HuggingGPT	6.7	79.2	–	–
LangChain Agent	7.9	77.0	–	–
TaskMatrix.AI	6.5	80.3	–	–
SAGE - ChatGPT-4	10.6	82.5 (+6.0)	–	–
Task 2: Multi-turn Dialog Completion (MultiWOZ)				
BabyAGI	8.2	–	73.5	–
AutoGPT	8.5	–	75.6	–
HuggingGPT	7.1	–	78.1	–
LangChain Agent	8.3	–	76.5	–
TaskMatrix.AI	6.8	–	79.9	–
SAGE - ChatGPT-4	10.9	–	81.6 (+5.8)	–
Task 3: Sequential Task with Dependencies (ALFWorld)				
BabyAGI	8.7	–	–	71.5
AutoGPT	8.3	–	–	73.1
HuggingGPT	7.5	–	–	75.2
LangChain Agent	7.8	–	–	72.8
TaskMatrix.AI	6.6	–	–	76.4
SAGE - ChatGPT-4	10.7	–	–	82.3 (+5.9)

D Examples

Cases study on TriviaQA in using GPT with SAGE.

User:

Answer the question based on the given passages. Only give me the answer and do not output any other words. The following are given passages:

History of canals in Great Britain - London Canal Museum History of canals in Great Britain Home > History > Canals in Britain In the history of canals, Britain was not a pioneer. The Chinese can claim that the Grand Canal of China was one of the first, in the tenth century, a.....

Answer the question based on the given passages. Only give me the answer and do not output any other words.

Question: Which metal is produced by the Bessemer Process?

Answer:

Assistant:

The content of the file seems to be an HTML document related to the administrative division of Turkey, including population statistics, charts, and maps. This file contains scripts, metadata, and structured data typical for a web page.

Checker:

Clear Objectives: When answering user questions, the focus should be on directly addressing the question without introducing irrelevant information. Document Handling: Before confirming whether the document content is relevant to the question, detailed aspects of the document should not be directly addressed.

Reflection:

Issue Description: The user wants to understand why the system outputs irrelevant HTML document content based on user input and seeks detailed feedback and reflection. Reasons for the Error: Misinterpretation of Input: The system mistakenly assumed that the user uploaded a file and attempted to read its content, whereas the user only provided text input. Processing Logic Error: Without clearly determining the user's intent, the system directly parsed and outputted file content. User Experience Problems: Irrelevant Output: The user expects concise, relevant answers, but the system provided lengthy and irrelevant HTML content, affecting the user experience. Poor Communication: Failure to accurately understand and respond to the user's real needs resulted in off-topic responses. In-depth Reflection and Improvement Suggestions: Clarify User Intent: Accurate Parsing of User Input: Enhance the system's natural language processing capabilities to accurately distinguish whether the user uploaded a file or simply provided text input. User Feedback Mechanism: Implement a user feedback mechanism to promptly adjust the understanding of user intent. For example, confirm the issue to verify if the user uploaded a file. Optimize Processing Logic: Layered Processing Flow: In system design, separate file processing logic from text response logic to avoid misjudgments. For instance, when a user uploads a file, process the file content separately; when a user inputs text, provide relevant answers directly. Pre-processing Check: Prior to handling files, perform pre-processing checks to confirm if the file content meets expectations.

LLMs with SAGE response: Steel.

Ground truth

Steel.

Cases study on LCC in using GPT with SAGE.**User:**

```
import networkx as nx
import pickle
from Queue import PriorityQueue
import copy
import random
import string
import sys
class MultiDiGraphEdgeKey(nx.MultiDiGraph):
    .....
```

Assistant:

You will be given the history of a past experience where you were given a task and failed to complete it. Reflect on the strategy and actions taken. Devise a concise, new plan of action that accounts for your previous mistakes. Here are two examples: Example 1: If you failed to remember to lock the door, plan to double-check the door each time. Example2: If you missed a deadline, plan to set reminders earlier.

Checker:

The given code is a part of the Camerini algorithm, which is used for ranking edges in a directed graph. This algorithm is particularly useful for problems involving the identification of optimal or near-optimal substructures, like arborescences or spanning trees, in complex networks. The algorithm has two main phases:

Collapsing Phase: This phase identifies and collapses cycles in the graph, creating supernodes. This simplification helps in managing cycles more efficiently and prepares the graph for the expansion phase.

Expanding Phase: After collapsing cycles into supernodes, this phase expands the supernodes back into their original structure, resulting in a ranked arborescence, where the edges are sorted or prioritized based on certain criteria.

LLMs with SAGE response:

```
Z.addedge(eu, ev, **data.copy())
self.init(graph, Y, Z)
A = best(root)
self.init(graph)
e, d = self.next(A, Y, Z)
w = self.getgraphscore(A) - d
if d != float('inf') else float('inf')
P.put((-w, e, A, Y, Z))
solutions.append(A)
return solutions
def bestbranching(self, root):
    return self.best(root)
def bestarborescencenorooted(self, root):
    return self.best(root)
def bestarborescencerooted(self, root):
    return self.best(root)
def getgraphscore(self, graph):
    return sum(data[self.attr] for u, v, data in graph.edges(data=True))
def findroots(self, graph):
    roots = [node for node, degree in graph.indegree() if degree == 0]
    return roots
    .....
```

Cases study on Repobench-P in using GPT with SAGE.**User :**

Please complete the code given below.

```
{src//kademlia//operation//KadRefreshOperation.java
public class KadRefreshOperation implements Operation
private final KadServer server;
private final KademliaNode localNode;
private final KademliaDHT dht;
private final KadConfiguration config;
public KadRefreshOperation.....
```

Assistant:

To devise a new plan of action, let's analyze the previous mistakes and come up with a solution: Mistake 1: In the KadRefreshOperation class, the execute() method calls the BucketRefreshOperation and ContentRefreshOperation without any error handling or exception handling. This can lead to unexpected errors and failures. Solution: Add proper error handling and exception handling in the execute() method of the KadRefreshOperation class. This will ensure that any errors or exceptions are properly handled and the execution can continue smoothly.....

Checker:

In the 'KadRefreshOperation' class, there is no error handling when the 'execute()' method calls 'BucketRefreshOperation' and 'ContentRefreshOperation'. Analysis: If 'BucketRefreshOperation' or 'ContentRefreshOperation' throws an exception, the entire 'execute()' method will fail, potentially causing the interruption of the entire system's refresh operation. In the 'StoreOperation' class, the 'execute()' method does not handle the case of existing content in the DHT. Analysis: The 'execute()' method always returns 1 as the number of storage nodes, which is clearly incorrect. The correct number of nodes should be returned based on whether the content already exists in the DHT. Solution: Before storing the content, check if it already exists. If it does, return 0; otherwise, store the content and return the actual number of storage nodes.

.....

LLMs with SAGE response:

```
private final transient KademliaRoutingTable routingTable; private final transient Kad-
Configuration config; private final transient MessageFactory messageFactory;
/* Scheduled refresh operation */ private final Timer refreshTimer;
/** Initialize the Kademlia system */ @param ownerId The ID for this owner * @param
nodeId The ID for this node * @param inetAddress The address for this node * @param
port The port for this node * @param config Configuration for the Kad system * *
@throws java.io.IOException */ public JKademliaNode(String ownerId, KademliaId
nodeId, InetAddress inetAddress, int port, KadConfiguration config) throws IOExcep-
tion { this.ownerId = ownerId; this.config = config;
/* Initialize the Node */ this.localNode = new Node(nodeId, inetAddress, port);
/* Initialize the Server */ this.server = new KadServer(localNode.getNodeId(), config);
.....
```